Code-Layout, Readability and Reusability

Date	03 October 2026
Team ID	2
Project Name	FitBuddy – AI Fitness Plan Generator using Gemini Models
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the FitBuddy codebase https://github.com/Chethana45/fitbuddy r structure, readability and reusability.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	PEP 8 style, 4-space indentation in all Python files
2	Proper File Structure	Files and folders are logically organized	Yes	app/ (one module per responsibility), app/templates, app/static, tests/, reports/
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	e.g. latest_plan, weight_kg, nutrition_tip, fallback_model
4	Function / Method Names	Functions are descriptively named	Yes	Brief's names kept: generate_workout_gemini(), update_workout_plan(), save_plan(), get_latest_plan()
5	Code Comments	Inline and block comments explain logic	Yes	Docstrings on modules and public functions; comments on safety rules and fallback logic
6	Modular Design	Code is split into reusable functions/modules	Yes	Routes, DB, schemas, AI client, each generator and demo data are separate modules
7	No Redundant Code	Duplicate or unused code is removed	Yes	Shared day-card markup in _plan.html; one shared Gemini client and call helper
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Typed AI errors → friendly page / HTTP 502; validation → 422; nothing saved on failure; retry + fallback model

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	gemini_client.py (structured-JSON call with retry & fallback)	Python, google-genai	Plan generation, plan revision, tip generation	High
2	schemas.py (WorkoutPlan, UserInput, FeedbackRequest)	Python, Pydantic	HTML routes, JSON API, Gemini response_schema, validation, tests	High
3	database.py CRUD helpers	Python, SQLAlchemy	All routes, admin dashboard, tests	High
4	_plan.html (day-card partial)	Jinja2, HTML/CSS	result.html, version view, admin dashboard	High
5	demo_data.py (deterministic fallback)	Python	Demo mode, tip fallback, pytest, Locust	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clear separation of web, AI, data and presentation layers
Readability	5	Descriptive names, type hints, docstrings
Reusability	4	Core client, schemas and partials reused; generators are project-specific by design
Documentation / Comments	5	README with architecture + ER diagrams, routes table, setup for Windows and Mac/Linux
Overall Score	4.75 / 5	Maintainable and easy to extend